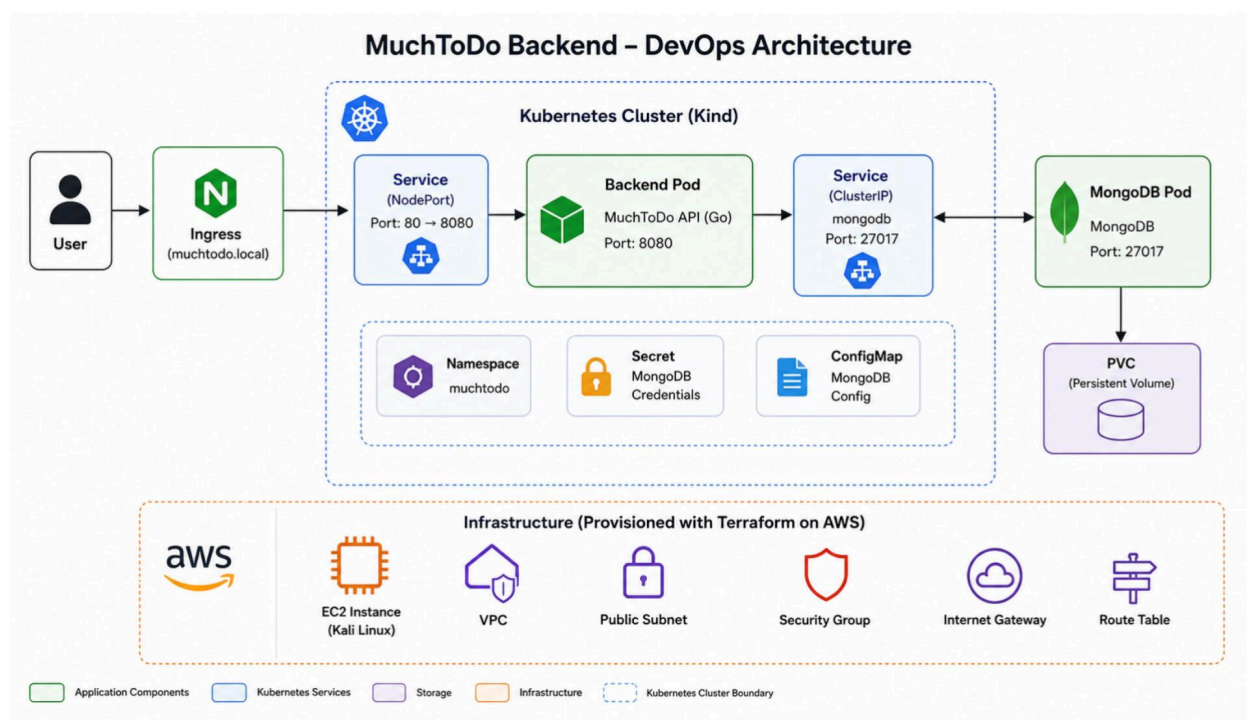


From Terraform to Kubernetes: Building and Deploying Backend Like a Real DevOps Engineer

Full End-to-End “MuchToDo” Project Documentation



Introduction

This project was a full hands-on DevOps assessment using a Backend API “MuchToDo”.

The goal was to move from infrastructure provisioning all the way to a working Kubernetes deployment using production-style best practices.

This included:

- AWS EC2 provisioning with Terraform modules
- Custom VPC and networking setup
- Docker installation and optimization
- Backend containerization
- Docker Compose services
- MongoDB persistent storage
- Kubernetes deployment using KIND
- Services and Ingress configuration
- Full API validation

This article documents exactly what was done, what failed, what was fixed, and what was learned.

PHASE 1 - Infrastructure Provisioning with Terraform

Objective

Provision a clean AWS environment instead of relying on default infrastructure.

What Was Built

Using Terraform modules:

- Custom VPC
- Public subnet
- Internet Gateway
- Route table
- Route table association
- Security Group

- Kali Linux EC2 instance
- Key pair access

Why This Matters

Senior engineers avoid unmanaged infrastructure.

Infrastructure as Code improves:

- repeatability
- scalability
- team collaboration
- disaster recovery
- production reliability

 Infra Setup main.tf

```
~/container-assessment/infra $ ls
main.tf      provider.tf      terraform.tfvars
modules      terraform.tfstate  variables.tf
outputs.tf   terraform.tfstate.backup
~/container-assessment/infra $ cat main.tf
module "vpc" {
    source = "./modules/vpc"

    project_name      = var.project_name
    environment       = var.environment
    vpc_cidr          = "10.0.0.0/16"
    public_subnet_cidr = "10.0.1.0/24"
    availability_zone = "eu-west-1a"
}

module "security_group" {
    source = "./modules/security-group"

    project_name = var.project_name
    environment  = var.environment
    vpc_id       = module.vpc.vpc_id
}

module "ec2" {
    source = "./modules/ec2"

    project_name      = var.project_name
    environment       = var.environment
    ami_id            = var.ami_id
    instance_type     = var.instance_type
    key_name          = var.key_name
    subnet_id         = module.vpc.public_subnet_id
    security_group_id = module.security_group.security_group_id
}
~/container-assessment/infra $ █
```

 Public IP output from Terraform

```

module.vpc.aws_subnet.public: Creation complete after 12s [id=subnet-06a8f71e381fb0c93]
module.vpc.aws_route_table_association.public: Creating...
module.ec2.aws_instance.kali: Creating...
module.vpc.aws_route_table_association.public: Creation complete after 1s [id=rtbassoc-0a01cd7773af3a04f]
module.ec2.aws_instance.kali: Still creating... [00m10s elapsed]
module.ec2.aws_instance.kali: Creation complete after 15s [id=i-0d5f112a3f5bf845c]

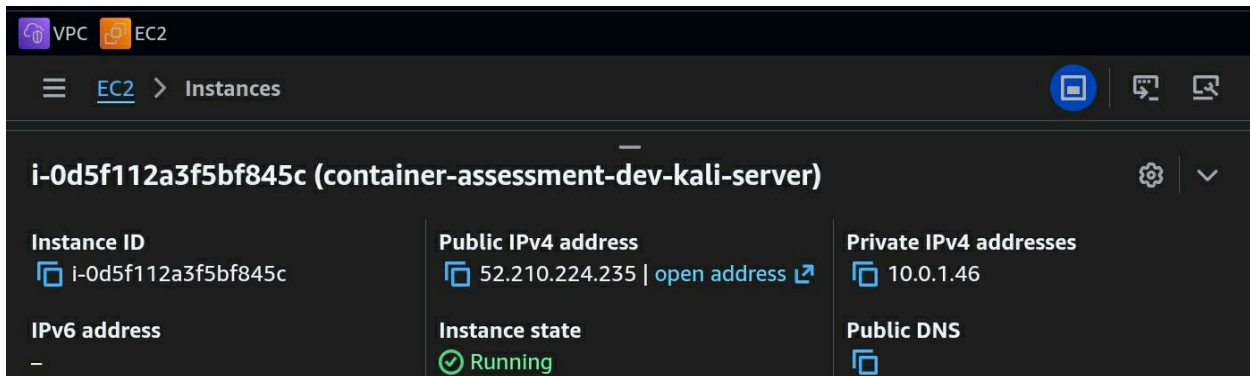
Apply complete! Resources: 8 added, 0 changed, 0 destroyed.

Outputs:

instance_id = "i-0d5f112a3f5bf845c"
instance_public_ip = "52.210.224.235"
ssh_command = "ssh -i ~/container-assessment/Adejare.pem kali@52.210.224.235"
~/container-assessment/infra $

```

 EC2 instance created successfully



The screenshot shows the AWS Management Console interface for the EC2 service. The breadcrumb navigation indicates the path is 'EC2 > Instances'. The instance 'i-0d5f112a3f5bf845c' is highlighted, with a title '(container-assessment-dev-kali-server)'. Below the title, a table displays the instance's details:

Instance ID I-0d5f112a3f5bf845c	Public IPv4 address 52.210.224.235 open address	Private IPv4 addresses 10.0.1.46
IPv6 address -	Instance state Running	Public DNS Public DNS

PHASE 2 - SSH Access and Server Validation

Objective

Connect to the EC2 instance and validate system readiness.

Validation Performed

Commands used:

- `whoami`
- `uname -a`
- `df -h`
- `free -h`

Key Discovery

The server had low memory for reliable Docker builds.

Fix Applied

Added 2GB swap memory.

This prevented future Docker build crashes.

Lesson

Senior DevOps work starts with environment stability.



SSH login successful

```
~/container-assessment/infra $ ssh -i ~/container-assessment/Ade  
jare.pem kali@3.248.224.157  
The authenticity of host '3.248.224.157 (3.248.224.157)' can't b  
e established.  
ED25519 key fingerprint is: SHA256:gpmY22TwEDEA4zTmFqLv4hAl/W825  
ly9Y3sNgEf/aWE  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprin  
t])? yes  
Warning: Permanently added '3.248.224.157' (ED25519) to the list  
of known hosts.  
Linux kali 6.18.12+kali-cloud-amd64 #1 SMP PREEMPT_DYNAMIC Kali  
6.18.12-1kali1 (2026-02-25) x86_64  
  
The programs included with the Kali GNU/Linux system are free so  
ftware;  
the exact distribution terms for each program are described in t  
he  
individual files in /usr/share/doc/*/copyright.  
  
Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
└─(Message from Kali developers)  
|  
| This is a minimal installation of Kali Linux, you likely  
| want to install supplementary tools. Learn how:  
| ⇒ https://www.kali.org/docs/troubleshooting/common-minimum-set  
up/  
|  
| This is a cloud installation of Kali Linux. Learn more about  
| the specificities of the various cloud images:  
| ⇒ https://www.kali.org/docs/troubleshooting/common-cloud-setup  
/  
|  
└─(Run: "touch ~/.hushlogin" to hide this message)  
└─(kali@kali)-[~]  
└─$
```

 Memory before swap


```

(kali㉿kali)-[~]
$ whoami
uname -a
df -h
free -h
kali
Linux kali 6.18.12+kali-cloud-amd64 #1 SMP PREEMPT_DYNAMIC Kali
6.18.12-1kali1 (2026-02-25) x86_64 GNU/Linux
Filesystem      Size  Used Avail Use% Mounted on
udev            954M   0  954M   0% /dev
tmpfs           194M 448K  194M   1% /run
/dev/nvme0n1p1   30G 898M   27G   4% /
tmpfs           968M   0  968M   0% /dev/shm
tmpfs           968M   0  968M   0% /tmp
none            1.0M   0   1.0M   0% /run/credentials/systemd-
journal.service
/dev/nvme0n1p15 124M 294K  124M   1% /boot/efi
none            1.0M   0   1.0M   0% /run/credentials/getty@tt
y1.service
none            1.0M   0   1.0M   0% /run/credentials/serial-g
etty@ttyS0.service
tmpfs           194M  4.0K  194M   1% /run/user/0
tmpfs           194M  4.0K  194M   1% /run/user/1000
                total        used        free      shared  buff/c
ache    available
Mem:      1.9Gi      251Mi      1.6Gi      456Ki      1
36Mi      1.6Gi
Swap:      0B          0B          0B

```

PHASE 3 - Server Dependencies Setups and Installation (Ansible)


```
└─$ cat setup.yml
---
- name: Provision Backend Environment (Kali Safe)
  hosts: localhost
  connection: local
  become: true

  vars:
    swap_file_size: 2G
    project_dir: /home/kali/much-to-do

  tasks:

    # -----
    # FIX DOCKER REPO ISSUE (KALI SPECIFIC)
    # -----
    - name: Remove broken Docker repo if present
      file:
        path: /etc/apt/sources.list.d/docker.list
        state: absent

    - name: Prevent Docker repo from being re-added
      copy:
        dest: /etc/apt/preferences.d/no-docker
        content: |
          Package: *
          Pin: origin download.docker.com
          Pin-Priority: -1

    - name: Update apt cache
      apt:
        update_cache: yes

    # -----
    # INSTALL SYSTEM DEPENDENCIES
    # -----
    - name: Install required packages
      apt:
        name:
          - docker.io
          - docker-compose
          - git
          - curl
        state: present

    # -----
    # ENABLE DOCKER
```

What Ansible Playbook (setup.yml) Did -

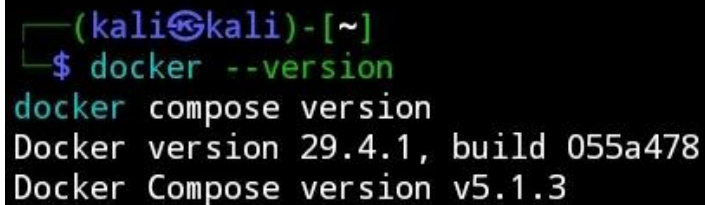
1. Fixes Kali + Docker repository conflict (Kali-specific)

- Deletes any broken `/etc/apt/sources.list.d/docker.list` file
- Creates a permanent apt pin (`/etc/apt/preferences.d/no-docker`) that blocks the official Docker repo forever
- Runs `apt update`

2. Installs system dependencies

- `docker.io` (Debian-packaged version - safe on Kali)
- `docker-compose`
- `git`
- `curl`

 `docker --version` output



```
(kali@kali)-[~]  
$ docker --version  
docker compose version  
Docker version 29.4.1, build 055a478  
Docker Compose version v5.1.3
```

3. Enables Docker properly

- Starts and enables the Docker service on boot
- Adds the kali user to the docker group (so you can run docker commands without sudo)

4. Creates 2 GB swap file (critical for builds)

- Checks if `/swapfile` already exists

```
    enabled: yes
    state: started

- name: Add kali user to docker group
  user:
    name: kali
    groups: docker
    append: yes

# -----
# SWAP (CRITICAL FOR BUILDS)
# -----
- name: Check if swapfile exists
  stat:
    path: /swapfile
    register: swapfile

- name: Create swap file
  command: fallocate -l {{ swap_file_size }} /swapfile
  when: not swapfile.stat.exists

- name: Set swap permissions
  file:
    path: /swapfile
    mode: '0600'
  when: not swapfile.stat.exists

- name: Format swap
  command: mkswap /swapfile
  when: not swapfile.stat.exists

- name: Enable swap
  command: swapon /swapfile
  when: not swapfile.stat.exists

- name: Persist swap in fstab
  lineinfile:
    path: /etc/fstab
    line: "/swapfile none swap sw 0 0"
    state: present

# -----
# PROJECT DIRECTORY SETUP
# -----
- name: Ensure project directory exists
  file:
    path: "{{ project_dir }}"
    state: directory
```

- If not: creates a 2 GB swap file, sets secure permissions, formats it, enables it, and adds it to /etc/fstab so it survives reboots

 Memory after swap enabled

```
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
free -h
Setting up swspace version 1, size = 2 GiB (2147479552 bytes)
no label, UUID=ed620349-dbb8-4b45-ae9f-e1cfe6dfae93
/swapfile none swap sw 0 0
      total        used        free      shared  buff/cache
Mem:   1.9Gi        250Mi       1.6Gi         456Ki         1
Swap:  2.0Gi          0B        2.0Gi
```

5. Sets up your project folder

- Creates the directory /home/kali/much-to-do
- Fixes ownership and permissions on the project folder and the entire /home/kali directory

PHASE 4 - Project Structure Inspection

Backend path:

~/Docker-k8-handson

Why This Matters

Wrong Docker build context causes broken deployments.

Engineers inspect first.

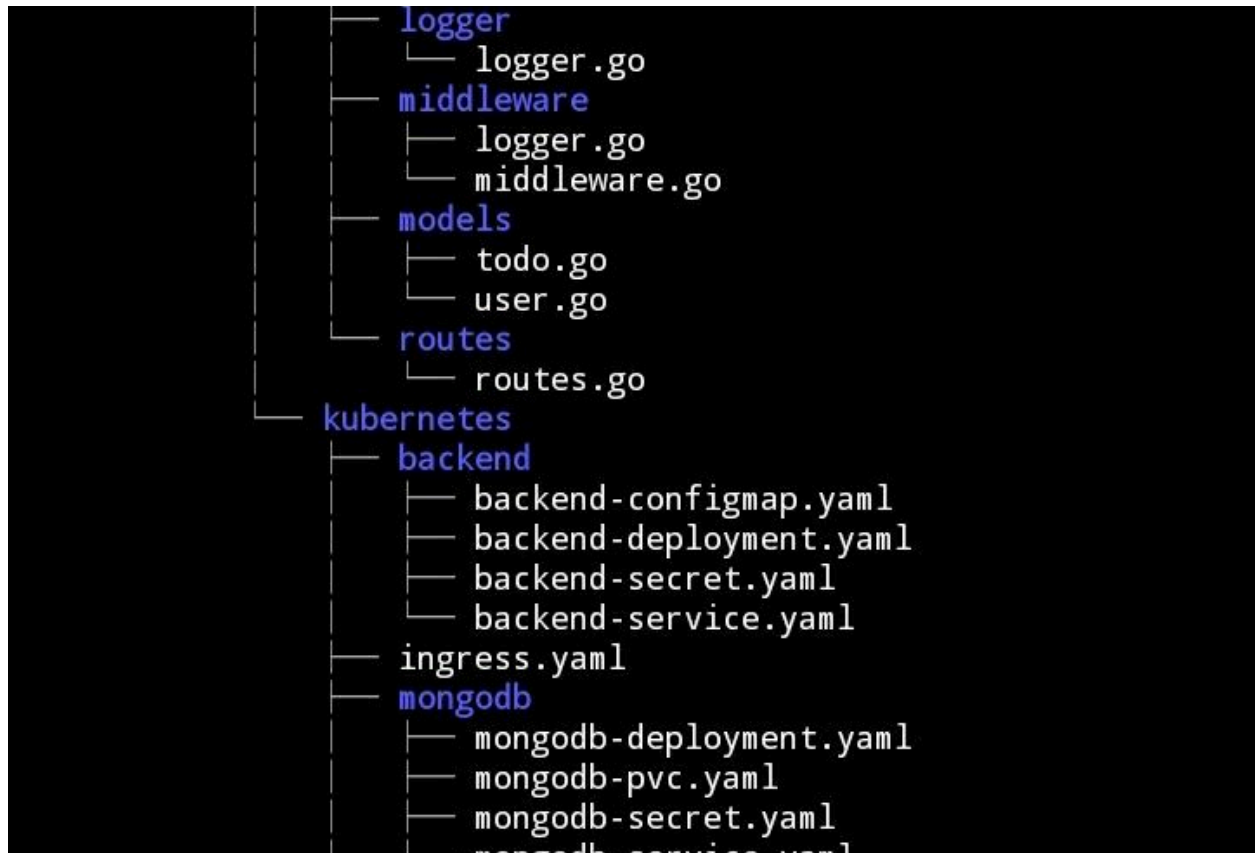
They do not assume.

 Project structure view

```
(kali㉿kali)-[~/Docker-k8-handson]
```

```
$ tree
```

```
├── ansible
│   └── setup.yml
├── infra
│   ├── main.tf
│   ├── modules
│   │   ├── ec2
│   │   │   ├── main.tf
│   │   │   ├── outputs.tf
│   │   │   └── variables.tf
│   │   ├── security-group
│   │   │   ├── main.tf
│   │   │   ├── output.tf
│   │   │   ├── outputs.tf
│   │   │   └── variables.tf
│   │   └── vpc
│   │       ├── main.tf
│   │       ├── outputs.tf
│   │       └── variables.tf
│   ├── outputs.tf
│   ├── provider.tf
│   └── variables.tf
├── much-to-do
│   ├── Server
│   │   └── MuchToDo
│   │       ├── Dockerfile
│   │       ├── Makefile
│   │       ├── cmd
│   │       │   └── api
│   │       │       └── main.go
│   │       ├── docker-compose.yml
│   │       ├── docs
│   │       │   ├── docs.go
│   │       │   ├── swagger.json
│   │       │   └── swagger.yml
│   │       ├── go.mod
│   │       ├── go.sum
│   │       ├── internal
│   │       │   ├── auth
│   │       │   │   ├── auth.go
│   │       │   │   └── auth_test.go
│   │       │   ├── cache
│   │       │   │   └── cache.go
│   │       │   ├── config
│   │       │   │   └── config.go
│   │       │   └── database
```



PHASE 5 - Phase 1 – Application Analysis

Key Findings

- Located Backend Entry point at:

cmd/api/main.go

I ran

go run cmd/api/[main.go](#)

- Go Version Mismatch

Problem:

go.mod required Go 1.25.1 but Dockerfile used Go 1.22

Fix

Updated Dockerfile builder image to supported Go version.

```
$ cat Dockerfile
FROM golang:1.25-alpine AS builder

WORKDIR /app
RUN apk add --no-cache git

COPY go.mod go.sum ./
RUN go mod download

COPY . .

RUN CGO_ENABLED=0 GOOS=linux GOARCH=amd64 \
    go build -ldflags="-w -s" -o server ./cmd/api

FROM alpine:3.19

RUN apk add --no-cache ca-certificates
RUN adduser -D appuser

WORKDIR /app

COPY --from=builder /app/server .

USER appuser

EXPOSE 8080

CMD ["/server"]
```

What Dockerfile Does

- Multi-stage build reduces image size
- Static binary compilation
- Environmental Configuration issue

Could not connect to MongoDB
error parsing uri: scheme must be "mongodb"

Root Cause

- `MONGO_URI` was empty
- Config loader (Viper) depended on env variables

Fix

```
export MONGO_URI="mongodb://localhost:27017"
```

2 - Docker Compose Optimization

```
$ cat docker-compose.yaml
services:
  mongodb:
    image: mongo:7
    ports:
      - "27017:27017"
    environment:
      MONGO_INITDB_ROOT_USERNAME: root
      MONGO_INITDB_ROOT_PASSWORD: example

  backend:
    build: .
    ports:
      - "8080:8080"
    environment:
      MONGO_URI: mongodb://root:example@mongodb:27017/?authSource=admin
    depends_on:
      - mongodb
```

```
docker compose up -d
curl http://localhost:8080/health
```

```
(kali㉿kali)-[~]
$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED
STATUS        PORTS
5223fc02ef05   muchtodo-backend                  "./server"              9 min
Up 9 minutes   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   muchtodo-backend-1
68befbbbed680   mongo:7                            "docker-entrypoint.s..." 9 min
Up 9 minutes   0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp   muchtodo-mongodb-1

(kali㉿kali)-[~]
$ curl http://localhost:8080/health
{"cache":"disabled","database":"ok"}
```

`{"cache":"disabled","database":"ok"}`

Phase 6 - Kubernetes Deployment with KIND

Backend Deployment

- Replica count: 2
- Container port: 8080
- Env vars injected

MongoDB Deployment

- Replica: 1
- Persistent Volume used

What Was Created

- Namespace
- Secret
- ConfigMap
- PVC

- MongoDB Deployment
- MongoDB Service
- Backend Deployment
- Backend Service
- Ingress Resource

Cluster Creation

kind create cluster --name muchtodo

```
(kali@kali)-[~/Docker-k8-handson/much-to-do/Server/MuchToDo]
$ kind create cluster --name muchtodo
Creating cluster "muchtodo" ...
  ✓ Ensuring node image (kindest/node:v1.35.0)
  ✓ Preparing nodes
  ✓ Writing configuration
  ✓ Starting control-plane
  ✓ Installing CNI
  ✓ Installing StorageClass
Set kubectl context to "kind-muchtodo"
You can now use your cluster with:

kubectl cluster-info --context kind-muchtodo

Thanks for using kind!

(kali@kali)-[~/Docker-k8-handson/much-to-do/Server/MuchToDo]
$ kind load docker-image muchtodo-backend --name muchtodo
Image: "muchtodo-backend" with ID "sha256:5a75981868060906153faef67373060287b3112f8277a8c7e4e39666f40719cf" not yet present on node "muchtodo-control-plane", loading...
```

...

```
(kali@kali)-[~/Docker-k8-handson/much-to-do/Server/MuchToDo]
$ kubectl get pods -n muchtodo
```

NAME	READY	STATUS	RESTARTS	AGE
mongodb-7c75b4b968-mvhq4	1/1	Running	0	75s

...

```

(kali㉿kali)-[~/much-to-do]
$ nano ~/much-to-do/kubernetes/mongodb/mongodb-secret.yaml

(kali㉿kali)-[~/much-to-do]
$ kubectl apply -f kubernetes/mongodb/mongodb-secret.yaml
secret/mongodb-secret created

(kali㉿kali)-[~/much-to-do]
$ kubectl get secrets -n muchtodo
NAME          TYPE      DATA   AGE
mongodb-secret  Opaque    2       19s

(kali㉿kali)-[~/much-to-do]
$ nano ~/much-to-do/kubernetes/mongodb/mongodb-configmap.yaml

(kali㉿kali)-[~/much-to-do]
$ kubectl apply -f kubernetes/mongodb/mongodb-configmap.yaml
configmap/mongodb-config created

(kali㉿kali)-[~/much-to-do]
$ kubectl get configmaps -n muchtodo
NAME          DATA   AGE
kube-root-ca.crt  1       3m54s
mongodb-config    1       7s

(kali㉿kali)-[~/much-to-do]
$ nano ~/much-to-do/kubernetes/mongodb/mongodb-pvc.yaml

(kali㉿kali)-[~/much-to-do]
$ kubectl apply -f kubernetes/mongodb/mongodb-pvc.yaml
persistentvolumeclaim/mongodb-pvc created

```

 kubectl get nodes

```

$ kubectl get nodes

NAME                                STATUS    ROLES    AGE     VERSION
muchtodo-control-plane             Ready    control-plane   5h34m   v1.35.0

```

 kubectl get namespaces

```
$ kubectl get namespaces
```

NAME	STATUS	AGE
default	Active	5h34m
ingress-nginx	Active	5h3m
kube-node-lease	Active	5h34m
kube-public	Active	5h34m
kube-system	Active	5h34m
local-path-storage	Active	5h34m
muchtodo	Active	5h22m

 kubectl get pvc -A

```
$ kubectl get pvc -A
```

NAMESPACE	NAME	STATUS	VOLUME
	CAPACITY	ACCESS MODES	STORAGECLASS
RIBUTESCLASS	AGE		VOLUMEATT
muchtodo	mongodb-pvc	Bound	pvc-d6a9de76-e8ab-443f-a54b-4
c09cbd8fe67	1Gi	RWO	standard
	5h19m		<unset>

 kubectl get pods

```
(kali@kali) - [~/Docker-k8-handson/much-to-do/Server/MuchToDo]
$ kubectl get pods -n muchtodo
```

NAME	READY	STATUS	RESTARTS	AGE
backend-8b848967b-5dg77	1/1	Running	0	52s
backend-8b848967b-csvgv	1/1	Running	0	52s
mongodb-7c75b4b968-mvhq4	1/1	Running	0	12m

PHASE 7 - Ingress Configuration / Validation

Ingress YAML Logic

- Host: `muchtodo.local`
- Path: `/`
- Service: `backend-service`

Added Domain Mapping in /etc/hosts

Mapped 127.0.0.1 <<>> muchtodo.local

```
GNU nano 8.7.1 /etc/hosts
# Your system has configured 'manage_etc_hosts' as True.
# As a result, if you wish for changes to this file to persist
# then you will need to either
# a.) make changes to the master file in /etc/cloud/templates/h
# b.) change or remove the value of 'manage_etc_hosts' in
#    /etc/cloud/cloud.cfg or cloud-config from user-data
#
127.0.1.1 ip-10-0-1-46.eu-west-1.compute.internal kali
127.0.0.1 muchtodo.local

# The following lines are desirable for IPv6 capable hosts
::1 localhost ip6-localhost ip6-loopback
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
```

```
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters

(kali@kali) - [~/Docker-k8-handson/much-to-do/Server/MuchToDo]
$ kubectl get ingress -n muchtodo
NAME                CLASS    HOSTS                ADDRESS    PORTS
AGE
muchtodo-ingress    nginx    muchtodo.local      localhost  80
5h22m
```

PHASE 8 - Ingress Errors and Fixes

Ingress Not Accessible

Symptoms

```
curl http://muchtodo.local/health
```

Error:

Could not connect to server

Root Cause Analysis

Layer	Status
Pods	OK
Services	OK
Ingress	OK
External Access	FAIL

Real Issue

Kind networking does NOT expose:

- LoadBalancer externally
- NodePort to localhost directly

Debugging Process

Step 1: Check ingress

```
kubectl get ingress -n muchtodo
```

Result:

ADDRESS: localhost

Step 2: Check ingress service

```
kubectl get svc -n ingress-nginx
```

Result:

LoadBalancer (EXTERNAL-IP: pending)

Get NodePort

```
kubectl get svc -n ingress-nginx
```

Output:

80:32404/TCP

Final Working Request

```
curl -H "Host: muchtodo.local" http://172.19.0.2:32404/health
```

```
(kali㉿kali)-[~/Docker-k8-handson]
$ curl -H "Host: muchtodo.local" http://172.19.0.2:32404/health
{"cache":"disabled","database":"ok"}

(kali㉿kali)-[~/Docker-k8-handson]
$ curl -H "Host: muchtodo.local" http://172.19.0.2:32404/ping
{"message":"pong"}

(kali㉿kali)-[~/Docker-k8-handson]
$ curl -H "Host: muchtodo.local" http://172.19.0.2:32404/
{"message":"Welcome to MuchToDo API"}
```

This confirmed:

- infrastructure working
- containers working
- storage working
- networking working
- services reachable
- Kubernetes deployment successful

...

9. Security Considerations

- Non-root container user
 - Environment-based secrets
 - Isolated network (Kubernetes)
-

10. Key DevOps Learnings

- Containerization is not enough → orchestration matters
 - Kubernetes networking requires layered debugging
 - Kind differs from cloud environments
 - Ingress requires proper exposure strategy
-

11. Final Status

- Docker build successful
 - Docker Compose working
 - Kubernetes deployment successful
 - Ingress configured
 - Application reachable via NodePort
 - MongoDB integration confirmed
-

Author
ADEJARE HASSAN